Course AI Assistant — Project Report

1. Team Members

Member	Full name	Matriculation number	Email
1	杜冰焱	2450319	2450319@tongji.edu.cn
2	黄茂军	2452776	Huang8888@tongji.edu.cn
3	李一芃	2454326	2454326@tongji.edu.cn
4	董泽福	2451201	2451201@tongji.edu.cn

2. Implemented Functionalities and Features

Course AI Assistant is an IntelliJ IDEA plugin that brings course-grounded Java assistance into the programming environment. Students can ask questions about lecture materials, include selected source code in a question, inspect supporting evidence, and request source changes that require explicit review and approval.

The project implements the Group C scope: F0 and F2. Its main contribution is the connection between traceable course knowledge and an IDE-native review workflow.

Requirement	Implemented behavior	Main implementation
F0: Course knowledge base	Registers PDF/PPTX files, extracts text by page/slide, splits knowledge chunks and builds an embedding index	`DocumentImporter`, `DocumentLoader`, `ChunkSplitter`, `CourseIndexer`
F0: RAG chatbot	Retrieves relevant chunks and sends the question, explicit code selection and evidence to an LLM	`LlmChatService`, `LocalKnowledgeSearch`
F0: Source citations	Validates citation identifiers; displays document and page/slide with expandable original evidence	`StructuredReplies`, `ChatAnswer`, `ConversationView`
F0: No relevant materials	Adds an explicit general-knowledge disclosure after successful retrieval returns no matches	`LlmChatService`
F0: Embedded interface	Provides Chat and Course materials in an IDEA Tool Window	`CourseAiToolWindowFactory`, `AssistantPanel`
F0: Code context	Adds the editor action “Ask Course AI Assistant” to capture the selected code	`AskAssistantAction`, `ContextCollector`

Requirement	Implemented behavior	Main implementation
F2: Instruction-driven changes	Generates a complete replacement for one Java project file from an explicit user instruction	`LiveModifyAction` , `LlmCodeGenerationService`
F2: Review before application	Displays original and proposed content through the built-in IntelliJ Diff API	`DiffReviewDialog`
F2: Explicit approval	Applies only an approved proposal after validation; cancellation preserves the original file	`ValidatedModificationService` , `FileSnapshotWriter`

Additional features include IDE-wide connection settings and a shared course library, separate Password Safe credentials for chat and embeddings, automatic indexing after adding materials, Markdown answer rendering, asynchronous work and cancellation, stale-proposal rejection, and undoable source editing.

2.1 Current Scope and Evidence

Production entry points are connected to the real-service adapters. Mock adapters and deterministic test clients support development and regression testing; they are not the production chat path. Historical design documents describe earlier project-local settings and separate retrieval previews. The current implementation uses global settings, a shared course library and evidence shown inside Chat.

On 3 November 2026, a clean local run passed **137 tests, 0 failures, 0 errors and 0 skipped tests across 22 suites**. Plugin packaging, project configuration verification and plugin structure verification also passed. The regression suite includes generated-file line-ending normalization and the handoff from LLM generation to proposal validation. These results do not establish live provider quality, real lecture retrieval accuracy or multi-window IDE acceptance.

Separately, Member 4 reported completing real-model F2 trials for a sum-method bug fix and a name-normalization method extraction, including Diff review, approved application, Undo and cancellation. The screenshots in Section 3.6 document the review and execution checks for these examples; they do not establish general model correctness.

3. User Manual

3.1 Installation and Preparation

For development, use JDK 21 and open the repository's `code/` directory as a Gradle project. The configured target is IntelliJ IDEA 2025.2.4. From `code/`, build the plugin using the supplied wrapper:

```
.\gradlew.bat buildPlugin verifyPluginProjectConfiguration verifyPluginStructure
```

The installable archive is generated in `build/distributions/`. Install it through IDEA's Plugins settings using “Install Plugin from Disk”, then restart if requested. Developers can instead launch the development IDE with `.\gradlew.bat runIde`.

3.2 Configure the Services

1. Open **Settings > Tools > Course AI Assistant > Language model**.
2. Enter a supported OpenAI-compatible base URL, model identifier and API key.
3. Open **Course RAG** and configure the embedding connection and its separate API key.
4. Save the settings. Saving settings alone does not call a provider.

These connections are shared by projects in the same IDE configuration. Keys are held separately in IDEA Password Safe rather than the ordinary settings XML. No provider or model is selected automatically.

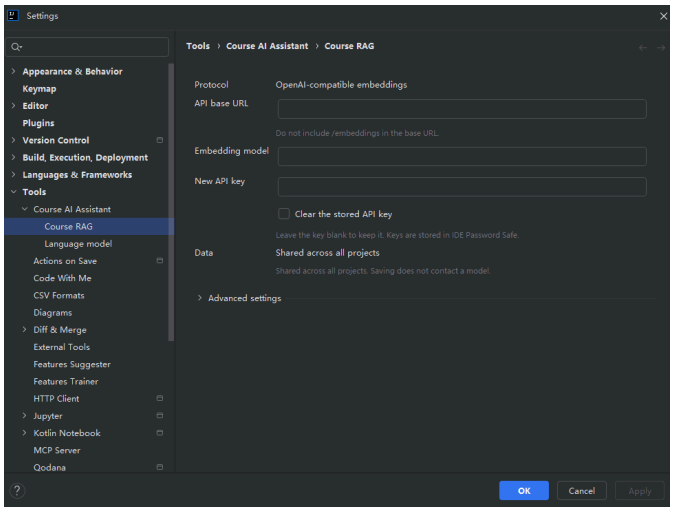


Figure 1 — Course RAG settings. Configure the embedding service URL, model and API key. Credential fields are empty in this screenshot.

3.3 Add Course Materials

Open the Course AI Assistant Tool Window and select **Course materials**. Click the plus icon and choose PDF/PPTX files. The plugin registers their original paths and starts indexing automatically when configuration is complete. A progress bar indicates ongoing work. After a failure, follow the message and retry. Files may be registered before credentials are complete, but usable indexing still requires the embedding service.

The library retains the original file paths without copying the source documents. Re-adding an existing path refreshes the index from the current file contents. Use a row's trash icon to remove a material from the library; this does not delete the original file. Removing all materials prevents Chat from using an obsolete index.

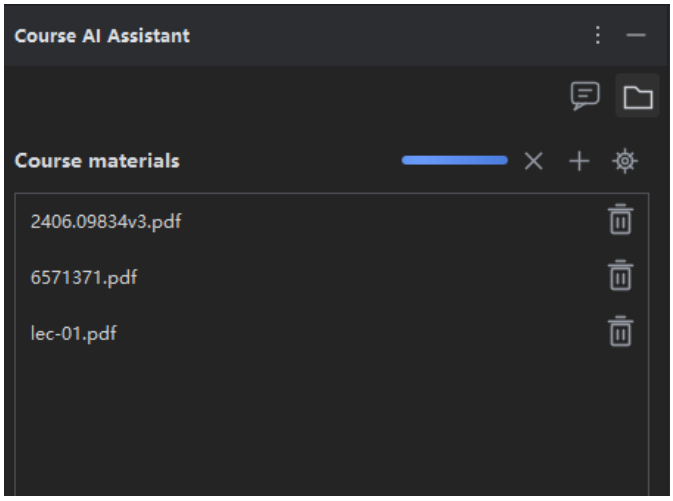


Figure 2 — Course materials and indexing progress. The panel lists registered documents and provides controls for adding and removing materials.

3.4 Ask a Course Question and Inspect Evidence

Select **Chat**, enter a question related to the imported lectures and press Enter. Use Shift+Enter for a new line. While the request runs, the conversation displays “Thinking”. Answers support Markdown, including code blocks and tables.

For a grounded answer, inspect the numbered source rows containing the document and page or slide. Click a row to expand the original evidence used for that answer. Expansion uses saved answer evidence and does not make another model or retrieval request. Citation numbering restarts at 1 for each answer.

If retrieval succeeds but finds no sufficiently relevant material, the answer explicitly states that it uses general knowledge. A retrieval failure produces an error instead of an apparently grounded answer.

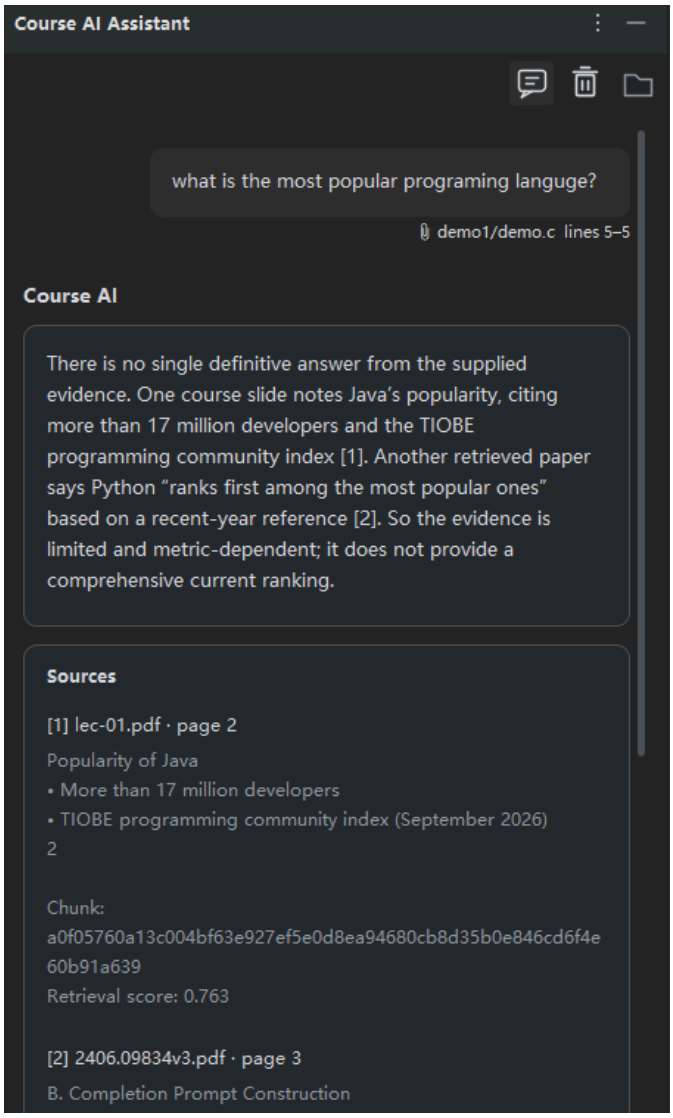


Figure 3 — Answer with source evidence. Numbered citations link the answer to document pages. Expanded sources show the original text and retrieval metadata.

3.5 Ask about Selected Code

Select a Java method or code segment in the editor. Right-click and choose **Ask Course AI Assistant**, then enter the question in Chat. The captured context includes the project-relative path, language, line range and selected text. Ordinary chat does not scan the whole project or send the entire file. Chat history is an in-memory display; previous turns are not sent as model conversation history.

3.6 Request and Review a Code Modification

1. Open a Java file inside the current project directory; selecting the relevant code is optional.
2. Right-click and choose **Modify Java Code with LLM...**.

3. Describe the intended change, for example: “Fix the sum method so that it correctly returns the sum of two integers. Preserve the class name, method signature, and all unrelated code.”
4. Review the summary and side-by-side original/proposed content in the built-in Diff Viewer.
5. Choose **Cancel - Keep Original** to reject, or **Approve and Apply** to apply.
6. Compile or run the affected example and inspect its behavior. Use Ctrl+Z in the editor to undo an applied change, then save if the restored version is required on disk.

Generating or previewing a proposal does not write the source file. Approval still requires a valid target and an unchanged original snapshot. If the file or relevant model configuration changes while the proposal is pending, generate a fresh proposal. Generated code is not automatically compiled or proven correct by the plugin.

The demonstration uses two cases: correcting `SumDemo.sum` from subtraction to addition, and extracting the name-trimming logic from `GreetingDemo.greet` into a private `normalizeName` method. Existing tests are preserved. Restore the original example before repeating a demonstration, because an unchanged proposal is rejected.

Files under `examples/f2/` are outside the default Java source sets. A self-contained example containing `main` can be compiled and executed with JDK 21 source-file mode, for example `java examples/f2/SumDemo.java` from `code/`. Successful completion of `runIde` does not verify an example's output. F2 uses the Language model connection and does not require course-material indexing.

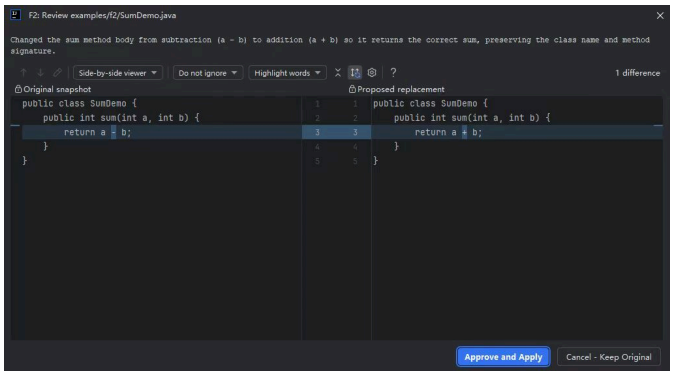


Figure 4(a) — Bug-fix review. The proposed change replaces subtraction with addition in `SumDemo.sum`. The built-in side-by-side Diff presents the original and proposed file, with explicit approval and cancellation controls.

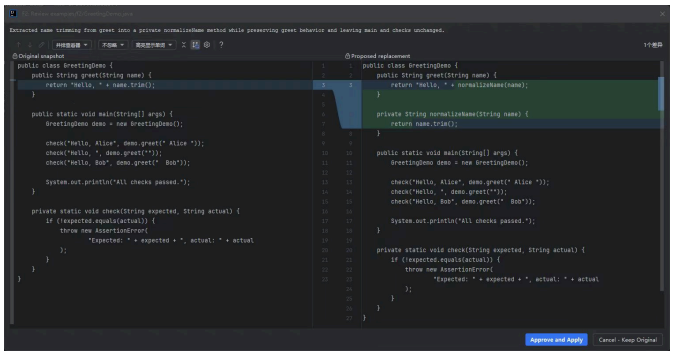


Figure 4(b) — Method-refactoring review. The proposal extracts `name.trim()` into a private `normalizeName` method and updates `greet` to call it, while preserving its signature. The existing `main` method and behavior checks remain unchanged.

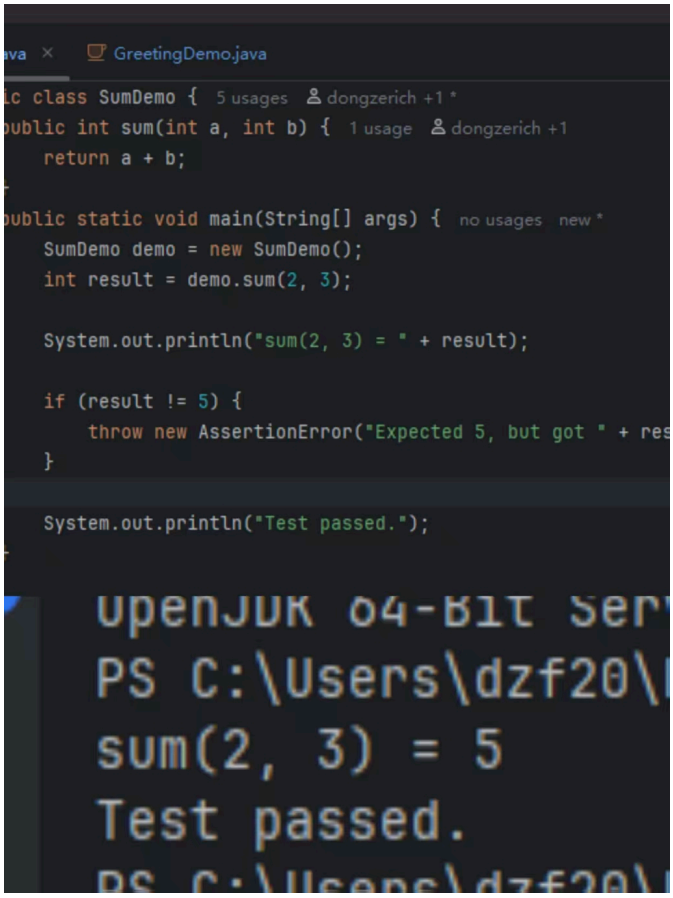


Figure 5(a) — Bug-fix execution check. The corrected example prints `sum(2, 3) = 5` and `Test passed.`, providing a behavior check separate from launching the plugin IDE.

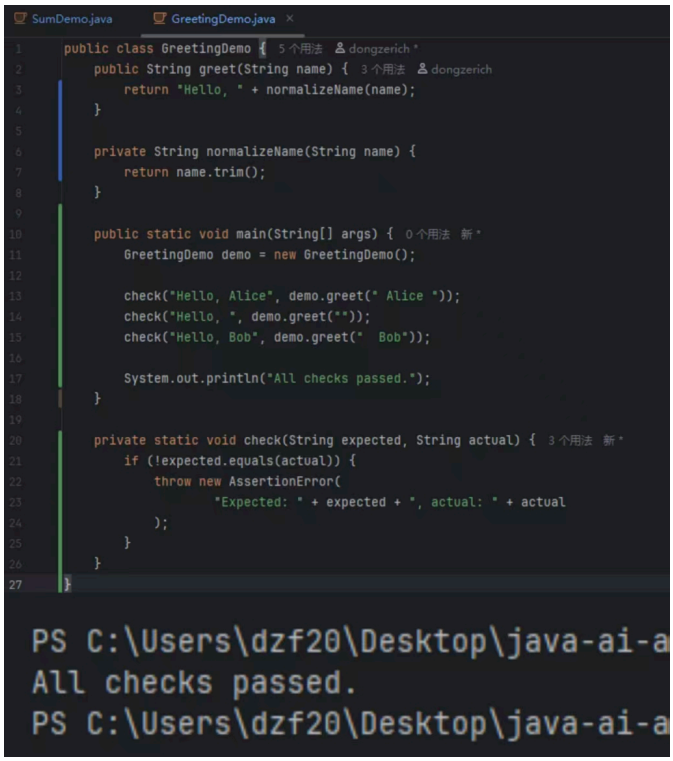
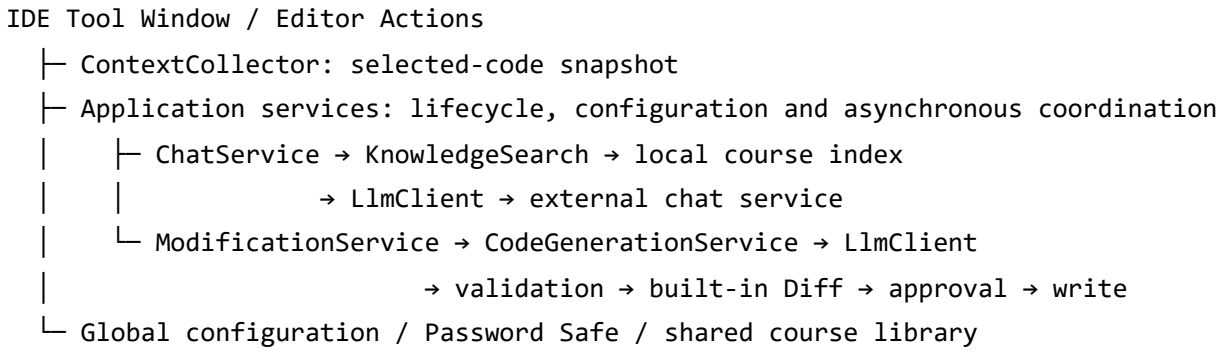


Figure 5(b) — Refactoring execution check. The saved code shows `greet` calling the extracted `normalizeName` method. The unchanged checks for ordinary, empty and whitespace-padded names complete with `All checks passed.`, confirming the expected behavior for these three inputs after refactoring.

4. Technical Design and Considerations

4.1 Architecture and Components

The plugin separates IDE presentation, application coordination, domain contracts and service adapters. UI components consume service results rather than constructing prompts or searching the vector store directly.



Package	Responsibility
`ui` , `action` , `context`	Embedded interaction, editor actions and immutable selected-code context
`application`	Project sessions, shared library access and request lifecycle
`model` , `service`	Data contracts and service interfaces
`rag`	Parsing, chunking, embeddings, local indexing and retrieval
`llm`	Prompts, HTTP transport, structured replies and failure mapping
`modification`	Proposal checks, review decisions and controlled source editing
`settings`	Global ordinary parameters and separate credentials

This structure allows retrieval and model behavior to be tested with controlled adapters, while IDE-specific behavior remains in the integration layer.

4.2 Overall Workflows

Indexing: register original file paths → extract page/slide text → form bounded chunks with source metadata → request document embeddings in batches → persist a validated vector snapshot → publish its manifest.

Question answering: capture question and optional selection → embed the question once → search stored document vectors → filter and rank chunks → send evidence and question to the LLM → validate the structured response and references → render answer and sources.

Modification: capture instruction, target and full-file snapshot → generate a single-file proposal → validate structure and target → open Diff → await explicit approval → revalidate current state → write through an undoable IDE command.

4.3 RAG Storage and Data Structures

The current library resides under the IDE configuration directory:

```
<IDE configuration>/course-ai-assistant/.course-ai/
  documents/<corpus-id>/documents.json
  indexes/<corpus-id>/manifest.json
  indexes/<corpus-id>/store-<generation-id>.json
```

Lock files coordinate registry and rebuild access. Course documents remain at their original locations. The registry contains normalized original paths and stable document identities. A stored vector snapshot contains chunk text, vectors and source metadata; querying an existing snapshot does not reparse the original documents.

Structure	Important information
`DocumentSection`	Document identity, page/slide kind and number, extracted text
`DocumentChunk`	Stable chunk ID, document ID, text and `SourceReference`
`SourceReference`	Reference ID, document name and PDF page range or PPTX slide scope
`KnowledgeChunk`	Retrieved text, source information and retrieval metadata
`IndexManifest`	Format/configuration fingerprints, dimensions, chunk count, source hashes, snapshot name and checksum
`ChatAnswer`	Answer content and the actual cited sources/evidence
`ChangeProposal`	Request/proposal identity, target path, original snapshot, replacement and summary

The manifest identifies the embedding model/input configuration and chunking strategy. Incompatible model, dimensions, prefix or chunking changes require rebuilding. Changing only the similarity threshold does not require new document embeddings.

New snapshots are written before the manifest is atomically replaced. A failed build does not publish a partial index. Retention of the previous disk snapshot is a recovery measure; it does not mean Chat can use stale evidence after the active library becomes invalid.

4.4 Document Parsing and Chunking

PDFBox extracts PDF text by physical page; Apache POI reads PPTX text by slide, including supported text shapes and tables. Optional slide notes can be included. The implementation does not provide OCR for scanned pages or text contained only in images.

The `boundary-codepoint-v2` splitter processes each page/slide independently. It first preserves whole paragraphs, then splits oversized paragraphs into sentences with Java `BreakIterator`, oversized sentences into whitespace-delimited words, and only indivisible oversized strings into Unicode code points.

Default chunk size is **800 Unicode code points**, with a maximum overlap budget of **120 code points**. Overlap reuses complete trailing units and reserves space for new content. Actual overlap may be smaller or zero; the algorithm avoids splitting surrogate pairs. These limits are character based, not token based.

Keeping each chunk within one page/slide preserves an unambiguous location. Boundary-aware splitting retains more meaningful text than fixed-width slicing. The trade-off is variable chunk lengths and occasional context loss at page boundaries.

4.5 Embeddings, Indexing and Retrieval

The embedding adapter sends batched requests to `<base-url>/embeddings`, using a configured model and separate credential. The default document batch size is 16. Responses are checked for input/output alignment, dimensions and valid vectors.

LangChain4j provides an in-memory vector store persisted as JSON snapshots. At query time, the service embeds the question once and searches cached document vectors. The chat path requests up to five results. Results below the configured minimum score are removed; remaining results are ordered by descending score with deterministic chunk-ID tie breaking.

The library score is:

```
score = (cosine_similarity(query, chunk) + 1) / 2
```

It lies in [0, 1] and is not a probability of correctness. The default threshold of 0.70 is an initial configuration, not an empirically validated accuracy claim. It must be calibrated using relevant and unrelated course questions with known expected sources.

For N indexed chunks of dimension d, exhaustive similarity search takes approximately O(Nd) work and O(Nd) vector storage, in addition to text and metadata. Read-only partitions allow cancellation checks during search; they are not an approximate nearest-neighbor index. This approach is simple for a modest course library, while large libraries would justify an ANN index or external vector database.

4.6 LLM Integration and Prompt Engineering

Chat and modification generation share global Language model settings. The HTTP adapter uses the OpenAI-compatible `/chat/completions` protocol with non-streaming responses. This is a specific protocol contract, not a guarantee of compatibility with every provider.

The chat request contains the current question, optional selected code and retrieved evidence serialized as JSON. The system prompt defines a Java teaching assistant, treats evidence and code as untrusted data, prohibits invented sources and requests a single JSON object:

```
{"answer":"Explanation ... [[cite:R-example]]", "citations":["R-example"]}
```

The service checks that references exist in the current evidence and that body markers agree with the citation list. Page numbers come from parser metadata rather than model-generated values. The UI maps internal identifiers to local numeric labels.

For successful empty retrieval, the service adds a fixed general-knowledge disclosure. Retrieval failure remains an error and does not silently trigger this branch. Reference validation establishes identifier consistency; it cannot prove that every natural-language claim is supported by its cited excerpt.

The modification prompt requests exactly one complete replacement file:

```
{"path":"src/Example.java", "proposedFileText":"complete file contents", "summary":"char
```

The prompt requires preserving unrelated code and prohibits claims that a file was written, approved or tested. Local checks reject malformed results, a different target, empty

replacement text and unchanged proposals. No silent truncation is used when the configured input budget is exceeded. Invalid structured output fails explicitly rather than automatically generating additional paid retries.

4.7 F2 Review, Validation and Source Application

F2 captures a full original file snapshot because a replacement proposal must be compared against an exact baseline. The review dialog embeds IntelliJ's built-in Diff components and makes both sides read-only. Approval and cancellation are explicit decisions.

Proposal identity, request identity, target and original snapshot are checked. Before application, IDE integration rechecks target availability and current state. `FileSnapshotWriter` compares the document modification stamp and full text against the original snapshot inside `WriteCommandAction`. The replacement is then written as an undoable command and saved through IDEA's document manager.

This prevents a previously generated replacement from overwriting intervening edits. A save failure is reported separately: the editor may already contain the approved change, so the user can save or undo it. The workflow currently handles one file, not repository-wide refactoring, file deletion or automated correctness proof.

Responsibilities are divided at the proposal boundary: the LLM service generates the replacement and explanation, while the modification module validates the proposal, presents the review and controls application. Model generation alone does not grant permission to write a source file.

4.8 Configuration, Concurrency and Efficiency

Ordinary settings and the course library are global to the IDE configuration; chat sessions and modification requests remain project-local. `GlobalRagLibrary` owns shared retrieval resources, while `ProjectRagService` tracks each project's requests. Closing one project does not close the shared application library.

Global model changes invalidate pending generation and old review sessions. RAG changes invalidate dependent chat work. Background execution keeps provider calls and indexing off the Swing event thread; UI updates return to the EDT. Request/session checks suppress obsolete callbacks after cancellation, settings changes or disposal.

Document vectors are reused between questions, saving repeated parsing and embedding work. Batch embeddings reduce request overhead, though they do not necessarily reduce

provider token charges. Retrieving a small evidence set bounds prompt size. The application enforces configurable time and character budgets, but character counts do not predict model tokens exactly. Cancelling a local HTTP operation cannot guarantee that a provider has stopped processing or charging an accepted request.

4.9 Supporting Technologies

Technology	Project configuration / role
Java / JDK	21; application implementation and toolchain
Gradle Wrapper	9.0.0; reproducible project build entry point
IntelliJ Platform Gradle Plugin	2.10.2
IntelliJ IDEA target	2025.2.4
Kotlin Gradle Plugin	2.1.20; build configuration includes JVM target 21
LangChain4j	1.7.1; local vector-store integration
Apache PDFBox / POI	3.0.8 / 6.0.0; PDF/PPTX extraction
Gson	2.13.2; structured JSON handling
CommonMark	0.30.0 with GFM tables; answer rendering
JUnit Jupiter	5.13.4; automated regression tests

5. Other Technical Information

5.1 Collaboration

The team divided development into four connected areas. The Git history records each member's module contributions and the subsequent integration work.

Team member	Main contributions
杜冰焱 (`napper-d`)	Plugin setup, Tool Window and editor actions, shared interface design, code review, and integration of retrieval, chat and modification services; unified model settings and improved the chat interface
黄茂军 (`huangmaojun666`)	Course RAG implementation, document retrieval, duplicate-file feedback and removal of redundant retrieval code
李一芃 (`Scilana`)	LLM chat and code-generation services, including the adapters consumed by the chat and modification workflows
董泽福 (`dongzerich`)	Reviewed single-file modification workflow, integration of LLM-generated proposals with F2 review and application, and manual bug-fix/refactoring demonstration trials

Development used feature branches, including `feature/m2-rag` , `feature/member3-llm-services` and `feature/member4-f2` . Shared interfaces defined the information exchanged between modules: selected-code snapshots, knowledge chunks and source references, chat answers, modification requests and change proposals. This allowed members to develop their components separately and connect them through agreed contracts.

Integration proceeded incrementally. The RAG module was merged and connected to the materials interface; the LLM and F2 branches were then merged, followed by staged work on connection settings, live chat, interface improvements and the shared model configuration for modifications. Cross-module work is also visible in the F2 integration branch, which incorporated the LLM service branch before connecting generation to reviewed changes. Stage and integration review documents recorded interface decisions, issues and verification results alongside the code.

5.2 Important Problems and Solutions

Problem	Implemented response	Remaining limitation
Source locations could be invented by the model	Preserve parser metadata and validate evidence identifiers	Semantic support still needs inspection
Fixed-width chunks can cut meaningful sentences	Paragraph/sentence/word boundaries with bounded overlap	No cross-page chunk context
A retrieval error could look like no matching material	Separate failed retrieval from successful empty results	Provider availability remains external
Late asynchronous replies can overwrite newer state	Request identity, cancellation and session invalidation	Remote work may continue after cancellation
A proposal could overwrite intervening edits	Snapshot/stamp checks immediately before writing	Users must regenerate after state changes
Each project previously required its own setup	Global settings, credentials and shared course library	Historical project data is not migrated automatically
Invalid model JSON can produce unusable output	Strict schema and proposal/reference checks	Explicit errors can require manual retry

5.3 Limitations and Future Work

Current limitations include no OCR, exhaustive local vector search, no persistent conversational memory, no streaming response display, single-file modification and no automatic compilation of proposed changes. Old vector generations are retained without automatic historical-snapshot cleanup. Provider compatibility and retrieval thresholds require real-service validation.

Possible future work includes measured threshold calibration, OCR for scanned slides, scalable vector indexing, bounded conversational memory, improved protocol adapters and

optional compilation checks before approval. These are future directions, not implemented features.